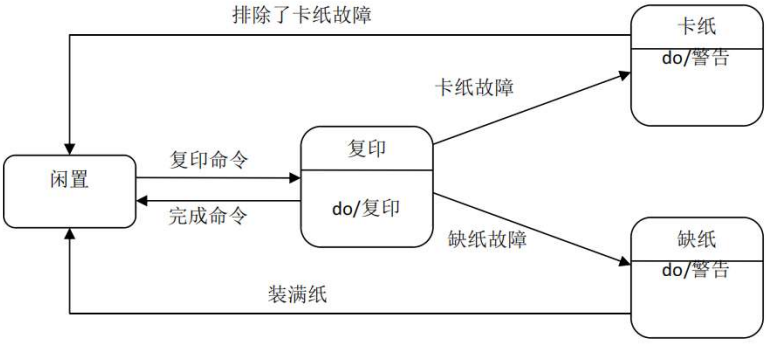
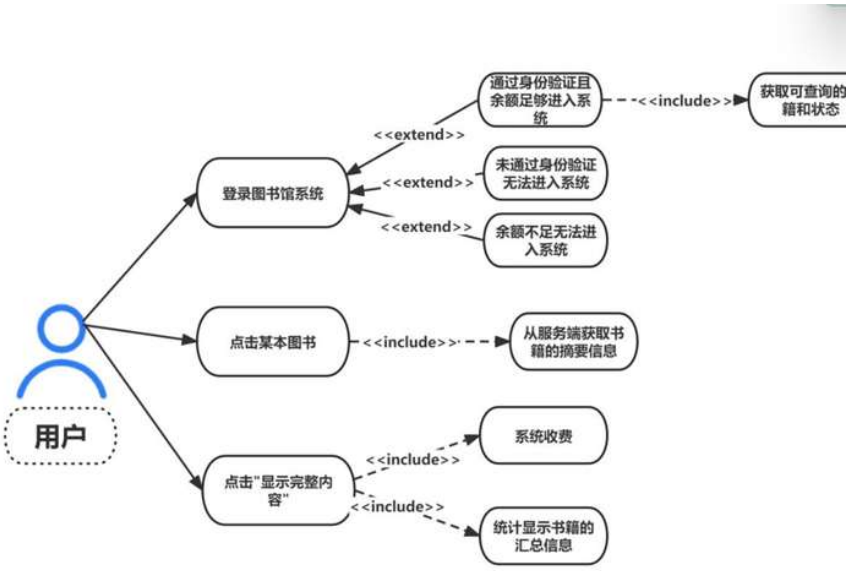
状态图



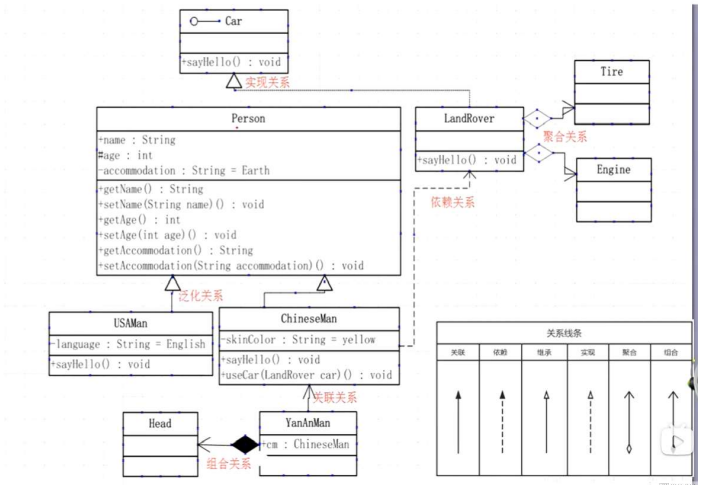
复印机的状态转换图

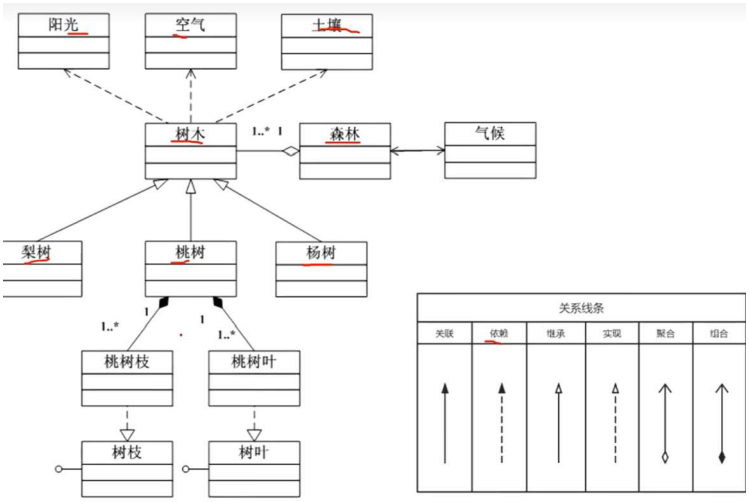
用例图



有属性人/信息的是实体类，人机交互/接口边界类 分配方法控制类

类之间关系实例一：



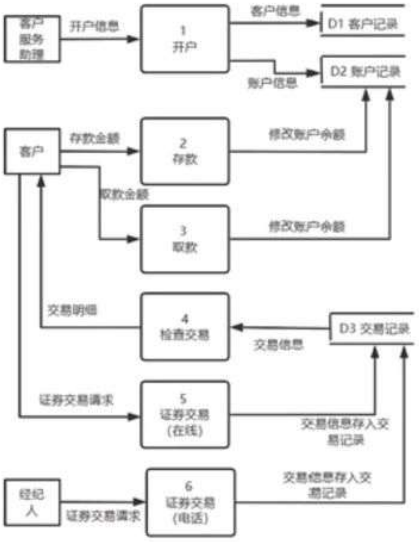


1. 数据流图的基本概念

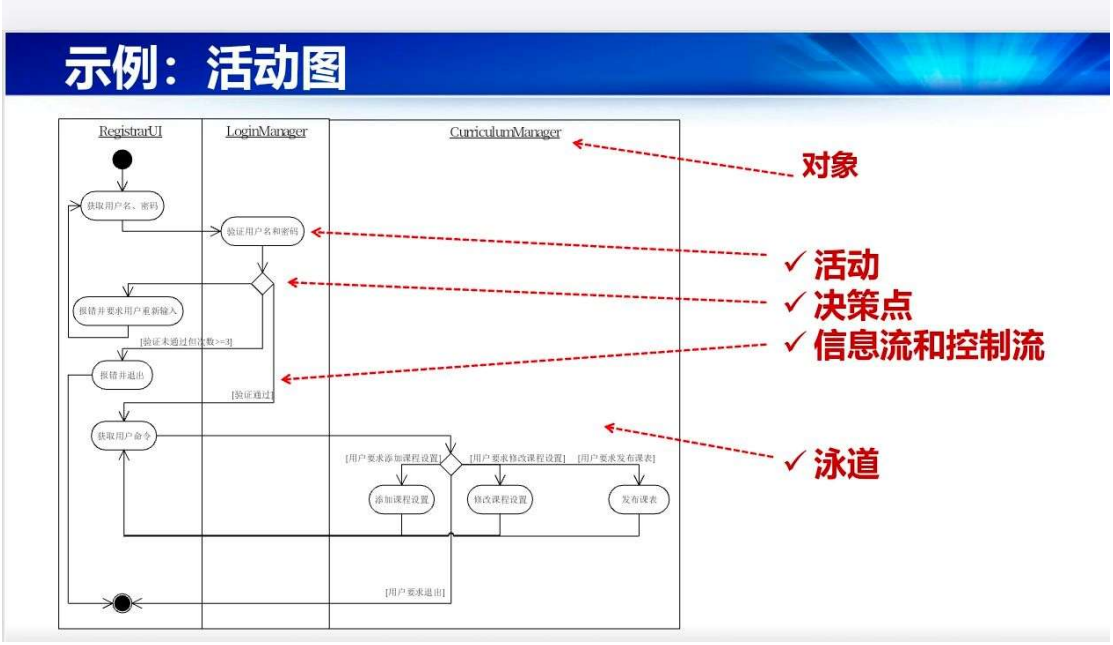
主讲



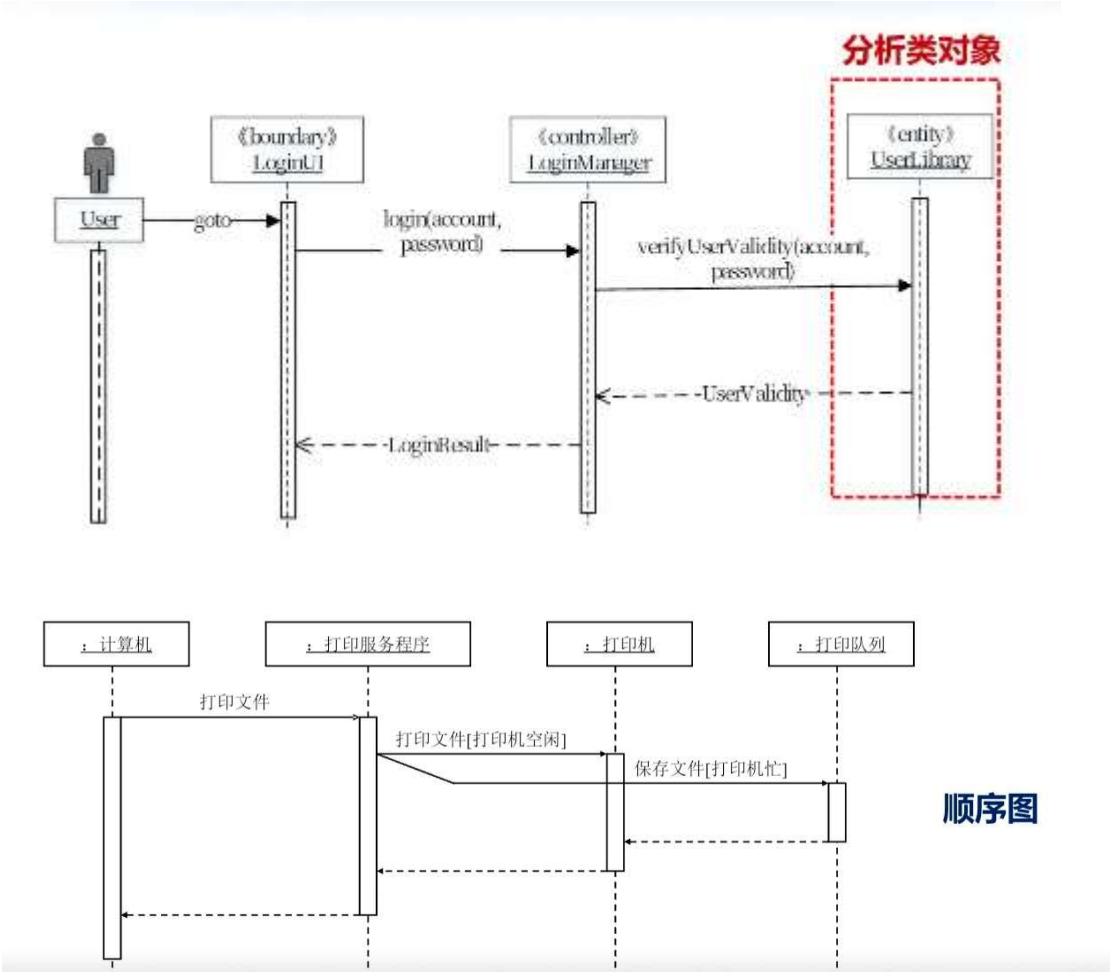
顶层图



0层图


示例：活动图
RegistrarUI
LoginManager
CurriculumManager
对象
✓活动
✓决策点
✓信息流和控制流
✓泳道

顺序图


分析类对象
《boundary》
LoginUI
《controller》
LoginManager
《entity》
UserLibrary
User
goto
login(account, password)
verifyUserValidity(account, password)
-UserValidity
-LoginResult
：计算机
：打印服务程序
：打印机
：打印队列
打印文件
打印文件[打印机空闲]
保存文件[打印机忙]
顺序图

优化设计类

示例：五子棋游戏中的设计类优化

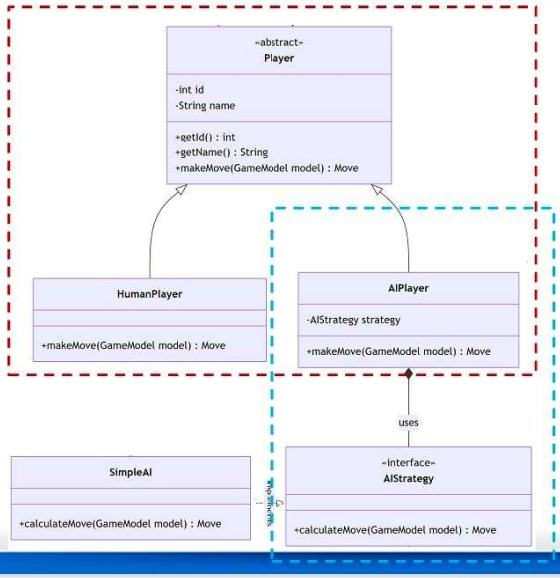
- 未优化前的AIPlayer类，将策略计算、执行都放在了方法makeMove中
- 缺点：
  - ✓每次新增策略都需要修改AIPlayer类，违反开闭原则。
  - ✓策略的计算和策略的执行紧密耦合在AIPlayer中，违反了单一职责原则。

1. Player（玩家）
   makeMove(position) 在指定位置下棋
2. AIPlayer（AI 玩家） 继承自 Player 类
   makeMove() ： AI 自动生成策略，并在合法位置下棋

未优化前的AIPlayer类

```
public void makeMove() {
    if ("random".equals(strategyType)) {
        makeRandomMove();
    } else if ("defensive".equals(strategyType)) {
        makeDefensiveMove();
    } else if ("offensive".equals(strategyType)) {
        makeOffensiveMove();
    }
    // 添加新策略？又来修改这个类！
}
```

示例：五子棋游戏中的设计类优化



(1) 优化Player、AIPlayer之间的关系

(2) 新增了策略类和接口，使用了依赖倒置原则

这样改变策略时，只需将不同策略strategy注入到AIPlayer中即可；